Reusable Agents in Claude Code

Build one once, invoke it forever, share it via GitHub

From the Claude Code Comprehensive Tutorial · github.com/hoqugr-beep/claude-code-tutorial

A **subagent** is a specialized assistant you define in a Markdown file. It runs in its own context window with its own system prompt, its own tool allowlist, and its own model. When you find yourself pasting the same long instructions into Claude Code over and over — "review this like a security engineer", "check my docs for broken links" — that's the signal to turn it into a subagent.

Three reasons to bother:

Benefit	Why it matters
Context isolation	The subagent burns its own context on greps and file reads, then hands you back only the summary. Your main conversation stays clean.
Enforced constraints	A reviewer with `tools: Read, Grep, Glob` *cannot* edit your code. That's a guarantee, not a request.
Sharing	Commit the file and your whole team gets the same agent. Package it as a plugin and anyone can install it.

---

1. The anatomy of an agent file

An agent is one Markdown file: YAML frontmatter for configuration, and the body becomes the system prompt.

```
---
name: docs-reviewer
description: Reviews Markdown docs for accuracy and broken links. Use proactively after editing
  docs/.
tools: Read, Grep, Glob, Bash
model: sonnet
color: purple
---

You are a technical documentation reviewer.

When invoked:
1. Run `git diff --name-only` to find which docs changed.
2. Read only the changed files.
3. Report findings and stop.
```

Only `name` and `description` are required. Everything else is optional.

> **The `description` field is not documentation**
>
> Claude reads `description` to decide *when to delegate to this agent automatically*. Write it as a trigger condition, not a summary. `"Reviews Markdown docs. Use proactively after editing docs/"` fires reliably; `"a docs helper"` does not.

2. Decide where the file lives — this is the "shareable" decision

Scope is determined entirely by which directory you drop the file into. Higher priority wins when two locations define the same `name`.

Location	Who gets it	Priority
Managed settings	Everyone in your org (admin-deployed)	1 (highest)
`--agents` CLI flag	Just this one session	2
`.claude/agents/`	Anyone who clones the repo	3
`~/.claude/agents/`	You, in every project on your machine	4
A plugin's `agents/` dir	Anyone who installs the plugin	5 (lowest)

Pick `.claude/agents/` when the agent is about this codebase. It gets committed to git, so your teammates receive it on their next `git pull` — no install step. This is the simplest sharing mechanism that exists.

Pick `~/.claude/agents/` when the agent is about how *you* work. Available in every project, never committed anywhere.

Pick a plugin when you want versioned distribution across many repos or to people outside your team. Covered in *section 6*.

Both directories are scanned recursively, so `agents/review/security.md` is fine — the subdirectory doesn't affect the agent's identity. Identity comes from the `name` field only. Keep `name` values unique across the whole tree.

3. Create it — two ways

Option A: ask Claude to write it (recommended)

Describe the agent and where it goes. Claude writes the frontmatter and prompt for you:

```
Create a docs-reviewer subagent in .claude/agents/ that reviews changed Markdown
files for accuracy, broken relative links, and stale version numbers. Make it
read-only and have it use Sonnet.
```

Option B: write the file yourself

```
mkdir -p .claude/agents
$EDITOR .claude/agents/docs-reviewer.md
```

Paste in the frontmatter-plus-prompt structure from section 1.

> **When you need to restart**
>
> Claude Code watches `.claude/agents/` and `~/.claude/agents/` and picks up new or edited files within a few seconds — no restart needed. **One exception:** the watcher only covers directories that existed at session start. If you just created the `agents/` directory for the first time, restart Claude Code once.

This repo ships a real, working example at `.claude/agents/docs-reviewer.md`. Clone the repo and it's immediately available — that's project-scope sharing in action.

---

4. Initiate it — four ways, escalating in force

This is the part people get stuck on. There is no "run agent" command; you invoke agents through the normal prompt.

4.1 Let Claude delegate automatically

Just do the work. If your `description` matches the situation, Claude delegates on its own:

```
> update the installation section in docs/index.md
# You edit, then Claude notices docs-reviewer's description says
# "use proactively after editing docs/" and delegates to it
```

This is why the `description` wording matters. Include **"use proactively"** to encourage it.

4.2 Name it in plain English

No special syntax. Just say the name:

```
> Use the docs-reviewer agent to check my changes
> Have the docs-reviewer subagent look at docs/index.md
```

Claude usually delegates. "Usually" is the catch — it's still Claude's decision.

4.3 @-mention it — this one is guaranteed

Type `@` and pick the agent from the typeahead:

```
> @"docs-reviewer (agent)" check the changes I just made
```

Or type the mention form directly — `@agent-` followed by the name:

```
> @agent-docs-reviewer check the changes I just made
```

While you type this form the typeahead shows *file* matches instead of agents. Ignore that; the mention still resolves on submit. For a plugin agent, use the scoped name: `@agent-my-plugin:docs-reviewer`.

An @-mention guarantees *which* agent runs. It does **not** control the prompt the agent receives — your full message still goes to Claude, which writes the agent's task prompt from it.

4.4 Run the entire session as that agent

`--agent` replaces the main thread's system prompt, tools, and model with the agent's:

```
claude --agent docs-reviewer
```

The agent name shows as `@docs-reviewer` in the startup header. This persists across `--resume`. To make it the default for every session in a project, set it in `.claude/settings.json`:

```
{
  "agent": "docs-reviewer"
}
```

The CLI flag wins if both are set.

4.5 Confirm it's actually loaded

If nothing seems to happen:

```
/context        # your agent should appear under "Custom Agents"
/doctor         # reports duplicate agent names in the same directory
```

If it's missing, check: does the `name` field contain a `:`? (Not allowed — it's reserved for plugin scoping, and the file silently won't load.) Did you just create the `agents/` directory? (Restart once.)

---

5. Tighten it up

Tool access

Omit `tools` and the agent inherits everything available to subagents. List tools explicitly to lock it down:

```
tools: Read, Grep, Glob        # read-only: physically cannot edit
```

```
disallowedTools: Write, Edit   # inherit everything, then subtract
```

A read-only reviewer is the single highest-value use of this field. It can't "helpfully" rewrite your code while you weren't looking.

> **Don't list `Skill` in `tools` to preload skills**
>
> Use the separate `skills:` field for that. And if no entry in your `tools` list resolves to a real tool, the agent fails to launch outright.

Model and effort

```
model: haiku      # sonnet | opus | haiku | fable | claude-opus-5 | inherit
effort: low       # low | medium | high | xhigh | max
```

`model` defaults to `inherit` (same as your main conversation). Routing a high-volume grep-and-summarize agent to `haiku` is a real cost lever.

Every frontmatter field

Field	Required	What it does
`name`	**Yes**	Lowercase-and-hyphens identifier. Can't contain `:`. Filename needn't match.
`description`	**Yes**	When Claude should delegate to this agent.
`tools`	No	Allowlist. Inherits all subagent tools if omitted.
`disallowedTools`	No	Denylist, subtracted from whatever was inherited or listed.
`model`	No	`sonnet`, `opus`, `haiku`, `fable`, a full ID, or `inherit`. Default `inherit`.
`permissionMode`	No	`default`, `acceptEdits`, `auto`, `dontAsk`, `bypassPermissions`, `plan`.
`maxTurns`	No	Hard cap on agentic turns before it stops.
`skills`	No	Skills to preload into context at startup (full content, not just the description).
`mcpServers`	No	MCP servers this agent can reach — by name or inline definition.
`hooks`	No	Lifecycle hooks scoped to just this agent.
`memory`	No	`user`, `project`, or `local` — enables cross-session learning.
`background`	No	`true` to always run as a background task.
`effort`	No	Overrides session effort level.
`isolation`	No	`worktree` runs the agent in a throwaway git worktree.
`color`	No	`red`, `blue`, `green`, `yellow`, `purple`, `orange`, `pink`, `cyan`.
`initialPrompt`	No	Auto-submitted first turn when run as the main agent via `--agent`.

> **Plugin agents lose three fields**
>
> `hooks`, `mcpServers`, and `permissionMode` are ignored when an agent loads from a plugin, for security reasons. If you need them, the file has to live in `.claude/agents/` or `~/.claude/agents/`.

Throwaway agents for testing

Define agents inline as JSON, no files touched. They exist for that session only:

```
claude --agents '{
  "quick-reviewer": {
    "description": "Fast read-only review of changed files.",
    "prompt": "You are a code reviewer. Report issues by priority. Never edit files.",
    "tools": ["Read", "Grep", "Glob"],
    "model": "haiku"
  }
}'
```

Note `prompt` here does the job the Markdown body does in a file. Handy in CI scripts and for iterating before you commit anything.

---

6. Share it widely: package it as a plugin

Committing to `.claude/agents/` shares with people who clone the repo. A **plugin** shares with anyone, versioned, installable in one command, and reusable across all their projects.

6.1 Build the plugin

```
mkdir -p my-agents/.claude-plugin my-agents/agents
cp .claude/agents/docs-reviewer.md my-agents/agents/
```

Create `my-agents/.claude-plugin/plugin.json`:

```
{
  "name": "my-agents",
  "description": "Documentation and review agents for MkDocs projects",
  "version": "1.0.0",
  "author": { "name": "Your Name" }
}
```

> **The number-one plugin mistake**
>
> Only `plugin.json` goes inside `.claude-plugin/`. The `agents/`, `skills/`, and `hooks/` directories go at the **plugin root**, next to `.claude-plugin/` — never inside it.

Layout check:

```
my-agents/
├── .claude-plugin/
│   └── plugin.json
└── agents/
    └── docs-reviewer.md
```

6.2 Test before you publish

```
claude --plugin-dir ./my-agents
```

Then confirm it loaded — `/context` should list it under Custom Agents, and it should appear in the `@` typeahead as `my-agents:docs-reviewer`. Run `/reload-plugins` after edits instead of restarting. Validate the structure with:

```
claude plugin validate ./my-agents
```

6.3 Publish it in a marketplace

A marketplace is just a repo with a `.claude-plugin/marketplace.json` catalog:

```
{
  "name": "my-plugins",
  "owner": { "name": "Your Name" },
  "plugins": [
    {
      "name": "my-agents",
      "source": "./my-agents",
      "description": "Documentation and review agents for MkDocs projects",
      "version": "1.0.0"
    }
  ]
}
```

`source` paths resolve relative to the marketplace root — the directory containing `.claude-plugin/`. Push it to GitHub, and your teammates run:

```
/plugin marketplace add your-org/your-marketplace-repo
/plugin install my-agents@my-plugins
/reload-plugins
```

Then they invoke it under its namespaced name:

```
> @agent-my-agents:docs-reviewer review my changes
```

A private GitHub repo works fine as a marketplace — that's how you keep an agent library internal to your team. To bump a release, change `version` in `plugin.json`; users only get updates when that field changes. Omit `version` entirely and every commit counts as a new version.

---

7. Design rules that actually matter

- **One agent, one job.** A "code-reviewer-and-test-writer-and-deployer" delegates unpredictably because its `description` matches everything. Split it.
- **Write the description as a trigger, not a title.** It's the only thing Claude reads when deciding to delegate.
- **Give reviewers no write tools.** Constraint by tool list beats constraint by instruction every time.
- **Tell the agent when to stop.** Agents don't know your intent. "Report findings and stop" prevents scope creep.
- **Specify the output format.** You're consuming its report in a summary — make it structured.
- **Commit project agents to git.** An agent nobody else has isn't a team standard.
- **Iterate in `.claude/`, graduate to a plugin.** Fast feedback first, packaging later.

---

8. Try it right now, in this repo

```
git clone https://github.com/hoqugr-beep/claude-code-tutorial
cd claude-code-tutorial
claude
```

Then:

```
> @agent-docs-reviewer review the current state of docs/
```

The agent came with the clone. No install, no configuration. That's the whole point.

---

9. Adding an agent to GitHub, and downloading one from GitHub

An agent is a single plain-text file. That makes GitHub the simplest distribution channel available — no marketplace, no plugin manifest, no install command.

9.1 Push an agent to GitHub

If the agent belongs to a project, it's already in the right place. Commit it:

```
git add .claude/agents/docs-reviewer.md
git commit -m "Add docs-reviewer agent"
git push -u origin main
```

Check your `.gitignore` first. A blanket `.claude/` entry is common and will silently drop the agent from the commit. If you have one, carve out an exception:

```
.claude/*
!.claude/agents/
.claude/settings.local.json
```

Verify before you push — `git check-ignore` tells you the truth:

```
git check-ignore -v .claude/agents/docs-reviewer.md   # no output = not ignored, you're fine
git status --short .claude/                            # should list the file
```

If you want a personal agent library that isn't tied to one project, make a dedicated repo:

```
mkdir claude-agents && cd claude-agents
git init
mkdir agents
cp ~/.claude/agents/*.md agents/
printf '# My Claude Code agents\n\nDrop these into `~/.claude/agents/`.\n' > README.md
git add -A
git commit -m "Initial agent library"
git branch -M main
git remote add origin https://github.com/YOUR-USER/claude-agents.git
git push -u origin main
```

Add a README that says what each agent does and which tools it holds. People decide whether to trust an agent from its README.

> **Let Claude do it**
>
> ```
> > commit the docs-reviewer agent and push it to a new public repo called claude-agents
> ```

9.2 Download a single agent from GitHub

The fastest path. Open the file on GitHub, click **Raw**, copy the URL, and curl it into place:

```
mkdir -p ~/.claude/agents
curl -fsSL -o ~/.claude/agents/docs-reviewer.md \
  https://raw.githubusercontent.com/hoqugr-beep/claude-code-tutorial/main/.claude/agents/docs-re
    viewer.md
```

Swap `~/.claude/agents/` for `.claude/agents/` to scope it to the current project instead. Raw URL anatomy:

```
https://raw.githubusercontent.com/<owner>/<repo>/<branch>/<path-to-file>
```

Or with the GitHub CLI, which works on private repos you have access to:

```
gh api repos/hoqugr-beep/claude-code-tutorial/contents/.claude/agents/docs-reviewer.md \
  --jq '.content' | base64 -d > ~/.claude/agents/docs-reviewer.md
```

Or just ask Claude:

```
> download the docs-reviewer agent from
  github.com/hoqugr-beep/claude-code-tutorial into ~/.claude/agents/
```

> **Read the file before you use it**
>
> An agent file is a system prompt plus a tool allowlist — and possibly `hooks` (shell commands that run automatically on tool events) and `mcpServers` (external servers it may reach). Downloading one is closer to installing a script than saving a document. Open it, read the frontmatter, and confirm the tool list matches what the agent claims to do before you invoke it. Be especially wary of `permissionMode: bypassPermissions` and any `hooks` block.
>
> Plugin-sourced agents are safer on this axis: `hooks`, `mcpServers`, and `permissionMode` are ignored entirely when an agent loads from a plugin.

9.3 Clone a whole repo of agents

If a repo is a library of several agents, clone it and copy what you want:

```
git clone https://github.com/YOUR-USER/claude-agents.git
cp claude-agents/agents/*.md ~/.claude/agents/
```

To stay up to date instead of taking a one-time snapshot, symlink the cloned directory and `git pull` when you want the latest:

```
git clone https://github.com/YOUR-USER/claude-agents.git ~/src/claude-agents
ln -s ~/src/claude-agents/agents ~/.claude/agents/shared
# later:
git -C ~/src/claude-agents pull
```

Because `~/.claude/agents/` is scanned recursively, agents inside the symlinked `shared/` subfolder load normally. The subfolder name doesn't change how they're invoked — identity comes from the `name` field only.

For a big monorepo where you only want the agents directory, use a sparse checkout:

```
git clone --filter=blob:none --sparse https://github.com/some-org/big-repo.git
cd big-repo
git sparse-checkout set .claude/agents
cp .claude/agents/*.md ~/.claude/agents/
```

9.4 Install from GitHub as a plugin

For versioned agents you don't want to copy by hand, point Claude Code at the marketplace repo and let it manage updates:

```
/plugin marketplace add YOUR-USER/claude-agents-marketplace
/plugin install my-agents@my-plugins
/reload-plugins
```

Private repos work the same way, as long as your git credentials can read them — this is the normal setup for keeping an internal team agent library. Users refresh the catalog with **`/plugin marketplace update`**.

Trade-offs versus copying a raw file:

	Raw file / clone	Marketplace plugin
Setup for the author	None — just commit	Needs **`plugin.json`** + **`marketplace.json`**
Updates for the user	Manual re-download or **`git pull`**	**`/plugin marketplace update`**
Versioning	Whatever the branch has	Pinned by **`version`** field
Agent name	Plain: **`@agent-docs-reviewer`**	Namespaced: **`@agent-my-agents:docs-reviewer`**
`hooks` / **`mcpServers`** / **`permissionMode`**	Honored	Ignored for safety

9.5 After downloading — confirm it loaded

```
ls ~/.claude/agents/          # file is there
claude                        # start (or restart) Claude Code
```

```
/context                      # agent listed under "Custom Agents"
> @agent-docs-reviewer review my recent changes
```

If it doesn't appear: you likely just created **`~/.claude/agents/`** for the first time, and the file watcher only covers directories that existed at session start — restart Claude Code once. Also check that the **`name`** field contains no **`:`** (reserved for plugin scoping; such files fail to load silently) and run **`/doctor`** to catch duplicate names.

Related sections

- Slash Commands & Skills — skills are model-invoked instructions; agents are separate contexts with their own tools
- Multi-Agent & Parallel Tasks — running several agents at once
- Permission Modes & Security — how `permissionMode` interacts with your settings